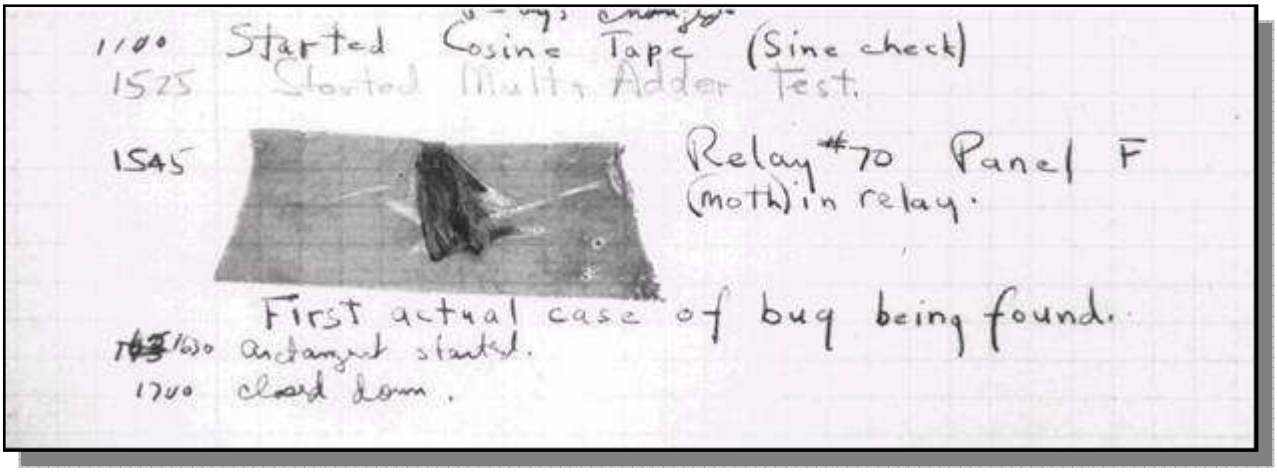
Homework 32

Assertions, Testing and Debugging

Based on Cay Horstman's Big Java Testing and Debugging Lab. Modified by Stephen Gilbert

When you hear the person sitting next to you in lab saying "I have a bug in my program", you don't have to worry about roaches infesting the computer. When we say *bugs* we really mean logical errors in our program. The first bug, however, was a real live bug. Well, live at first. In 1947 at Harvard University, a moth was found in one of the components of the Mark II computer, and was causing problems.



And so, the word *bug* was expanded to include not only insects, but also logical errors in computer programs. Evidently, the verb *to debug* became one of the most used verbs in the hi-tech world. In this homework assignment you will deal with debugging. You will practice locating and fixing some common types of bugs. The high point of the lab is a tour of the debugger inside the Eclipse environment.

Reading Stack Traces

Exercise 1
In Eclipse, create a new project named DebugLab. Describe exactly the steps you performed to complete these steps. Supply a screenshot of your new, empty Eclipse project.

Next, download the files **WordAnalyzer.java** and **WordAnalyzerTester.java** from the assignment page and import them into your Eclipse project.

Exercise 2
Describe the steps you performed to import the files into your project. Include a screenshot showing the Package Explorer window with both source files displayed.

Have a look at the `WordAnalyzer` class in the Eclipse editor. A `WordAnalyzer` is constructed with a string—the word to be analyzed. Right now, we only care about the first (buggy) method: `firstRepeatedCharacter`. It returns the first repeated character in a word, such as `o` in `roommate`.

The `WordAnalyzerTester` program simply tests the `WordAnalyzer` class.

Exercise 3

Without actually running the program, predict its output by reading both the comments and the code. Assume that the `firstRepeatedCharacter` method works correctly.

Exercise 4

Now run the program. What output do you get? (Use a screenshot to show me both the correct outputs and the error message.)

The program dies with an exception and prints a *stack trace*:

```
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: String index
out of range: 4
        at java.lang.String.charAt(String.java:558)
        at WordAnalyzer.firstRepeatedCharacter(WordAnalyzer.java:26)
        at WordAnalyzerTester.test(WordAnalyzerTester.java:14)
        at WordAnalyzerTester.main(WordAnalyzerTester.java:7)
```

Have a look at the stack trace. The first complaint is about the method `charAt` of the `java.lang.String` class. That's a library class. It is extremely unlikely that a library class has a bug. It is far more common that one of its methods was called with bad parameters.

The next complaint is about line 26 of the `firstRepeatedCharacter` method of the `WordAnalyzer` class.

Exercise 5

Exactly what source code do you find in line 26 of `WordAnalyzer`?

Now look at the call to `charAt` in line 26.

```
word.charAt(i + 1)
```

Exercise 6

In theory, there are two different exceptions that can be thrown in this call. What are they?

Of course, we know that `word` is not `null` in this case, so we know that the error must be the index.

Exercise 7

(a) Explain the nature of the bug, and how to fix it. (b) What output did you get after you fixed the bug?

Assertions

Have another look at the `WordAnalyzer` class. Suppose that some knucklehead passed a `null` pointer to the constructor:

```
WordAnalyzer wa = new WordAnalyzer(null);
```

This makes no sense, but if programmers always wrote code that makes sense, you wouldn't be working through this lab.

Exercise 8

Try it out. Modify `WordAnalyzerTester` and add a call `test(null)`. Where is an exception thrown? Why not in the constructor? (Print a screenshot)

It would be nice if we could get an exception thrown in the constructor. The easiest way is to use an *assertion*. An assertion is a condition that needs to be true for a program to function properly. If it is not, then it is better to abort the program with an error message. Here is how we would add an assertion to the `WordAnalyzer` constructor:

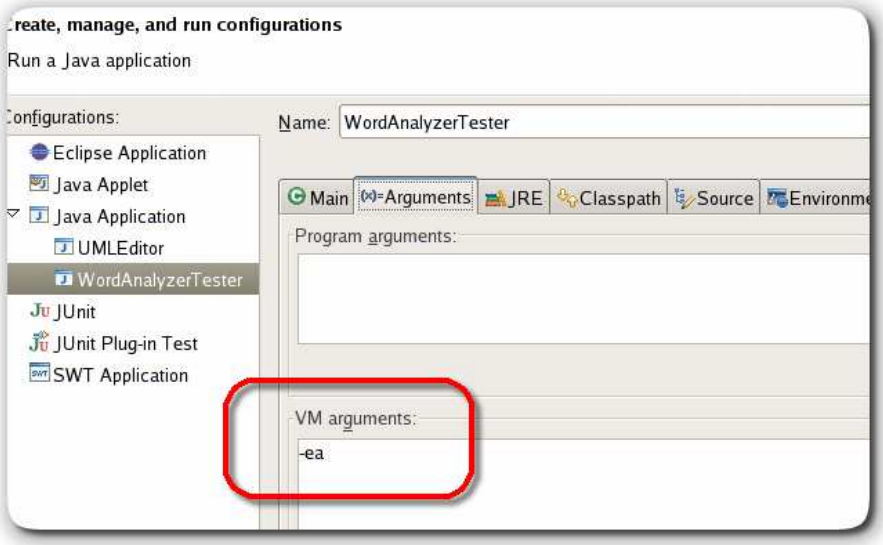
```
/**
   Constructs an analyzer for a given word.
   @param aWord the word to be analyzed (not null)
*/
public WordAnalyzer(String aWord)
{
   assert aWord != null;
   word = aWord;
}
```

Note that `assert` is a keyword (similar to `return` or `throw`), and not a method. Therefore, it is not necessary to use `()` around the condition.

Assertions are disabled by default, for greater efficiency. It takes a small amount of time to check each assertion, and in a well-tested application, it is not necessary to test conditions that aren't going to happen.

For testing and debugging, we want to turn assertions on.

In Eclipse to enable assertions, select the **Run -> Run...** menu item, then click on the **Arguments** tab. In the **VM arguments** field, add `-ea` as in the picture here



Exercise 9
Run the modified `WordAnalyzerTester` (that is, the version to which you added the `assert` statement) with assertions enabled. What is the stack trace now? Why is that more helpful? (Add a screenshot)

Using assertions doesn't fix your bugs. It simply gives a more accurate report about the causes of errors. Still, assertions are an easy and painless way for testing against bad things.

Exercise 10
Can you think of a situation in a recent homework assignment where you could use an assertion for something **other than** a null pointer check.

	Enabling assertions during testing only and leaving them disabled after testing is not a good default. It is like wearing a life vest while your boat is safely docked and flinging it over the railing once you are at sea. It would be better if `-ea` was the default for the virtual machine.

Logging

Exceptions and assertions are useful to pinpoint drastic errors that kill your program. But often, a sick program doesn't die. It limps along and computes the wrong result. In order to find out what such a program does, students often sprinkle their code with print statements like this one:

```
System.out.println("Yoohoo! I got this far!");
```

	Print statements are really dumb. You put them in, you take them out, you put them back in, you comment them out, you remove the comments, and when it all works you take them out for good. Until the next bug appears.

In this section, you will practice the use of *logging* statements. Logging is easy. Instead of using the `System.out` object, simply create a variable that refers to the default `Logger` object like this:

```
Logger logger = Logger.getLogger(Logger.GLOBAL_LOGGER_NAME);
```

and then call the logger's `info()` method in place of `println()` like this:

```
logger.info("About to return from find. i=" + i);
```

Logging is better than `System.out.println` for two reasons:

- There is **no shame** in logging. You can leave the logging statements in your code when you turn it in, demonstrating good software engineering practice.
- It is easy to turn all logging statements on or off.

Download and import the file **WordAnalyzerTester2.java** from the assignment page. Then open and examine the program in the Eclipse editor. The program tests the `firstMultipleCharacter` method of the WordAnalyzer class. That method also has a bug.

Exercise 11

Which input to the `firstMultipleCharacter` method does not yield the expected result?

We will use logging to find the problem. Create a `Logger` instance variable named `logger`, (using the instructions above), and then add the following statements at appropriate places of the `find` method:

```
logger.info("Entering find. c=" + c + ",pos=" + pos);
logger.info("About to return from find. i=" + i);
```

The `Logger` class is in the `java.util.logging` package.

Exercise 12

What is the code of your `find` method? Either copy and paste or shoot a screenshot.

Exercise 13

What output do you get when you run the `WordAnalyzerTester2` class? Show me a screenshot.

Exercise 14

Look at the logging messages. Using the information provided, explain why the `firstMultipleCharacter` method does not work.

Exercise 15

You can fix the problem by modifying either the `firstMultipleCharacter` method or the `find` method. Fix the problem. What fix did you make? Show me the result.

Exercise 16

Run the program again. What output do you get now?

Of course, now you no longer need the logging messages. To turn them off, add this statement to the `main` method in `WordAnalyzerTester2`:

```
logger.setLevel(Level.OFF);
```

Exercise 17

Add that statement and run the program again. What output do you get now?

Exercise 18

Suppose you find another bug and want to turn logging back on. How do you do that?

In a small program, using `logger.info` works fine. As your programs grow larger, you can control your logging in more sophisticated ways. You can use different logging levels. Instead of `info`, call methods `severe` for important messages or `fine` for "fine-grained" messages. Then call the `setLevel` method to set the level of the messages that you want to see in a particular program run. You can also define your own logger objects in addition to using the global `Logger`. Look at the API of the Logger class for more information.

Running a Debugger

Most development environments, including Eclipse, have a *debugger*, a tool that lets you execute a program in slow motion. You can observe which statements are executed and you can peek inside variables.

Debuggers can be complex, but fortunately there are only three commands that you need to master:

1. Set breakpoint
2. Single step
3. Inspect variable

In this assignment, you'll use the debugger that is a part of Eclipse. Begin by downloading the WordAnalyzerTester3.java file from the class Web page. This class tests the buggy method named `countRepeatedCharacters`. That method is supposed to count the substrings consisting of repeated character in a word. For example, the word `mississippiii` should have a count of 4.

Exercise 19
Run `WordAnalyzerTester3`. What output do you get? (Shoot me a screenshot.)

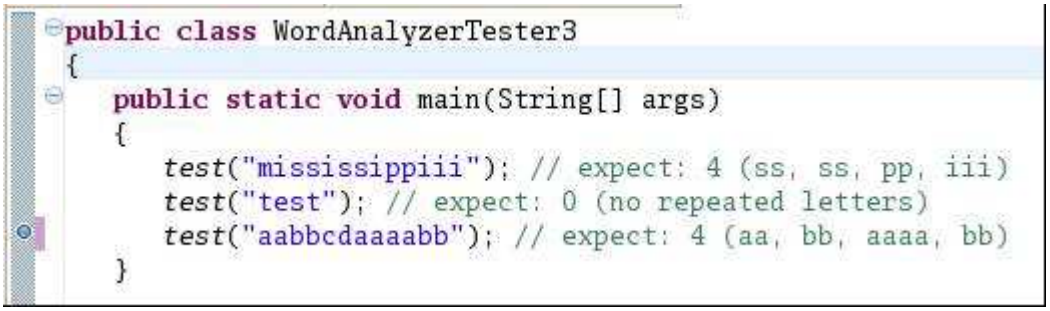
The `countRepeatedCharacters` method does the right thing for the first two test cases, but it mysteriously fails on the string `"aabbcdaaaabb"`. It should report 4 repetitions, but it only reports 3.

Setting Breakpoints

Unfortunately, the debugger cannot "go back", so you can't simply go to the point of failure and backtrack. Instead, you first run your program at full speed until it comes close the point of failure. Then you slow down and execute small steps, watching what happens.

We know that the first two calls to the `test` method in `WordAnalyzerTester3` give the correct results. It won't be too interesting to debug them. Instead, let's go directly to the third call. We'll set a *breakpoint* at that line, so that the debugger stops as soon as it reaches the line.

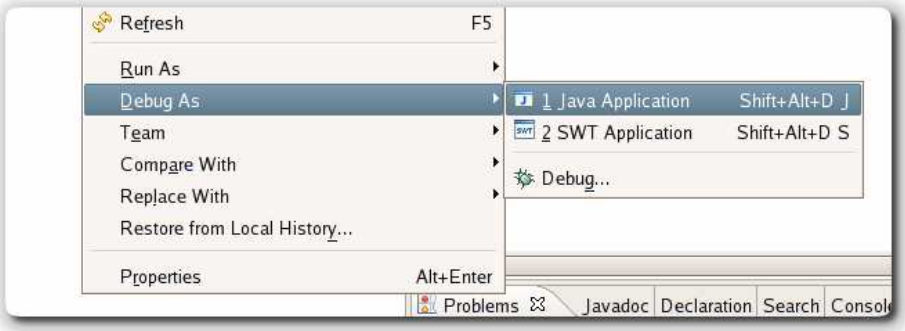
Move the cursor to the third call to `test`. Then select **Run -> Toggle Line Breakpoint** from the menu. You will see a small blue dot to the left of the line, indicating the breakpoint.



```
public class WordAnalyzerTester3
{
   public static void main(String[] args)
   {
      test("mississippiii"); // expect: 4 (ss, ss, pp, iii)
      test("test"); // expect: 0 (no repeated letters)
      test("aabbcdaaaabb"); // expect: 4 (aa, bb, aaaa, bb)
   }
```

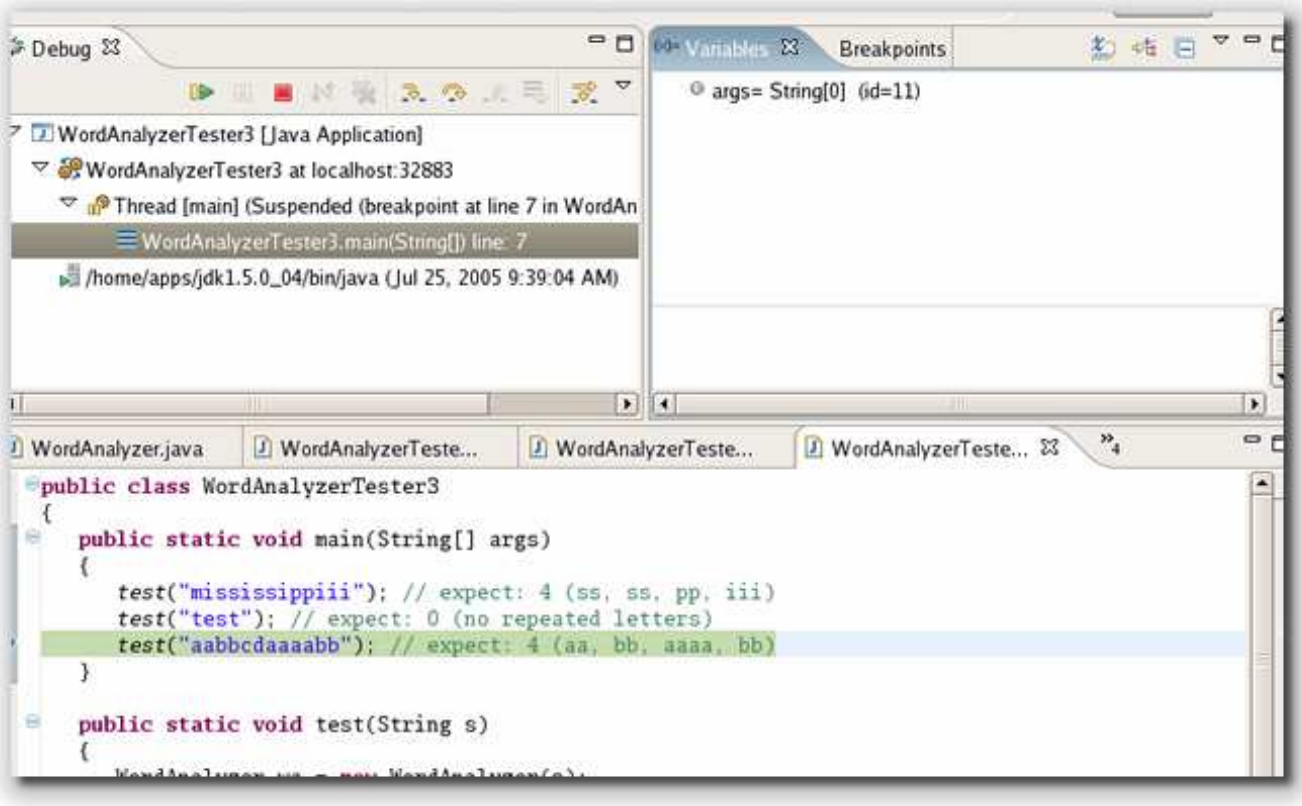
Now right-click on the **`WordAnalyzerTester3`** class in the package explorer window.

Select the menu option **Debug As -> Java Application**.



The debugger now runs your program. When it hits the first breakpoint, you get this dialog:

Simply click the **Yes** button, and you'll see a wholly different set of windows (i.e. the Debug perspective):



The green line shows where the debugger has stopped.

Exercise 20
Launch the debugger as described. What output do you get in the console window? Why do you get two lines of output and not three?

Single stepping

Now we want to step through the `test` method in slow motion. Single-stepping is such a common command that Eclipse gives you three ways of executing it:

1. Select **Run -> Step Into** from the menu
2. Hit the F5 key
3. Click the "step into" icon in the Debug window

Exercise 21
Execute the "Step Into" command. What happens?

Now you are inside the `test` method. The next call is

```
WordAnalyzer wa = new WordAnalyzer(s);
```

That call is boring, and we do not want to step into the constructor. We'll use the "Step Over" command instead.

Exercise 22
What are the three ways for executing the Step Over command in Eclipse?

Exercise 23
Execute the "Step Over" command. What happens?

Now you are at the line

```
int result = wa.countRepeatedCharacters();
```

Exercise 24
Remember, we want to find out why we get the wrong repetition count for the third test case. Should you execute "Step Into" or "Step Over" at this point? Why?

Execute "Step Into" a couple of times. You should get into the lines

```
int c = 0;
```

and

```
for (int i = 1; i < word.length() - 1; i++)
```

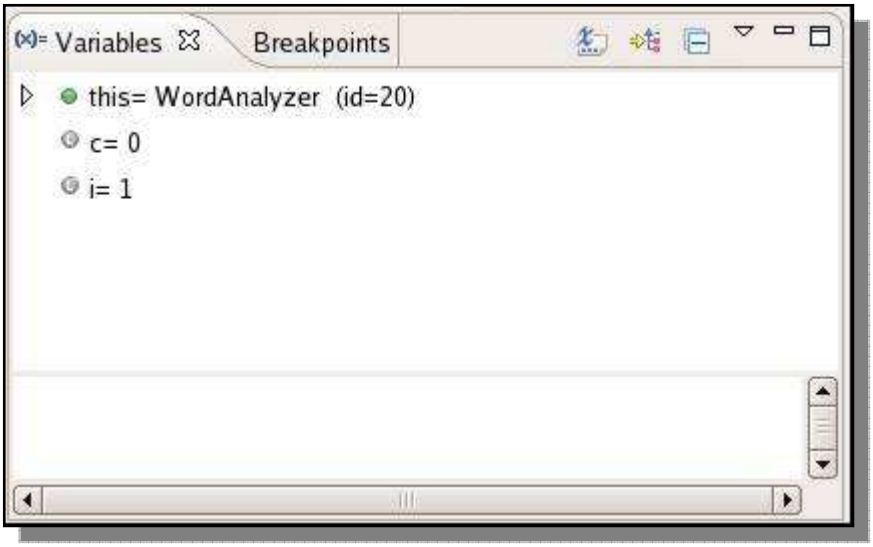
Now when you execute "Step Into" again, you will end up in the code for the `String` class. That sometimes happens by accident. In this case, there is no harm done since the `length` method is so short. But it can be

bothersome to be trapped in a long library method (such as `println`). The remedy is the "Step Return" command. It gets you out of the current method and back to the caller.

Exercise 25
As described, execute "Step Into" until you are inside the `length` method of the `String` class. Then execute "Step Return". What happens?

Inspecting Variables

Look inside the Variables window in the top right corner.



You can see the contents of three variables, `this` (the implicit parameter of the call `wa.countRepeatedCharacters()`), `c`, and `i`.

The triangle next to `this` indicates that you can expand the variable.

Exercise 26
Click on the triangle next to `this`. What do you see?

Exercise 27
Why are there no triangles next to `c` and `i`?

Now keep executing the Step Over command. Watch what happens to the `c` and `i` values. You'll see `i` increase each time the loop is executed.

Exercise 28
The value of `c` increases three times. What are the values for `i` at each increase?

Exercise 29

Look at the `this.word` value. What is special about the three positions at which `c` increases?

We will fix the bug in the next section. For now, select the **Run -> Resume** menu option. Your program runs again at full speed, until it hits the next breakpoint or until it terminates.

Exercise 30

What happens when you execute the "Resume" command?

You now know how to use the debugger. You have learned how to set breakpoints, how to single-step, and how to view variables. The Eclipse debugger has many advanced commands, but they are not needed for simple programs.

Fixing the bug

The `countRepeatedCharacters` method looks for character sequences of the form `xyy`, that is, a character followed by the same character and preceded by a different one. That is the *start* of a group. Note that there are two conditions. The condition

```
if (word.charAt(i) == word.charAt(i + 1))
```

tests for `yy`, that is, a character that is followed by another one just like it. But if we have a sequence `yyyy`, we only want it to count once. That's why we want to make sure that the preceding character is different:

```
if (word.charAt(i - 1) != word.charAt(i))
```

This logic works almost perfectly: it finds three group starts: `aabbcdaaaabb`

Exercise 31

Why doesn't the method find the start of the first (`aa`) group?

Exercise 32

Why can't you simply fix the problem by letting `i` start at 0 in the `for` loop?

Exercise 33

Go ahead and fix the bug. (a) What is the code of your `countRepeatedCharacters` method now? (b) Run the `WordAnalyzerTester3` method again. What is the output now?

Is the program now free from bugs? That is not a question the debugger can answer. As the famous computer scientist Edsger Dijkstra pointed out: "*Program testing can be used to show the presence of bugs, but never to show their absence!*" As you have seen in this lab, testing and debugging is a laborious activity. In your computer science education, it pays to pay special attention to the tools and techniques that ensure correctness without testing.